

ADAPTIVE TIME-STEPPING FOR MASSIVELY-PARALLEL HARDWARE-ACCELERATED ROBOT SIMULATION

Marco DiGiorgio and Vince Kurtz

DePaul University

THE PROBLEM

Massively parallel GPU simulators power modern robot learning, but every world runs a **single fixed timestep**. Small δt wastes work everywhere to satisfy the stiffest contact; large δt produces penetration, jitter, and non-physical artifacts **that learned policies exploit, breaking sim-to-real transfer**.

Error-controlled integration already solves this by varying δt with system stiffness. But existing controllers are **single-world and CPU-bound**: they assume one trajectory at a time, not thousands of worlds stepping in lockstep on a GPU.

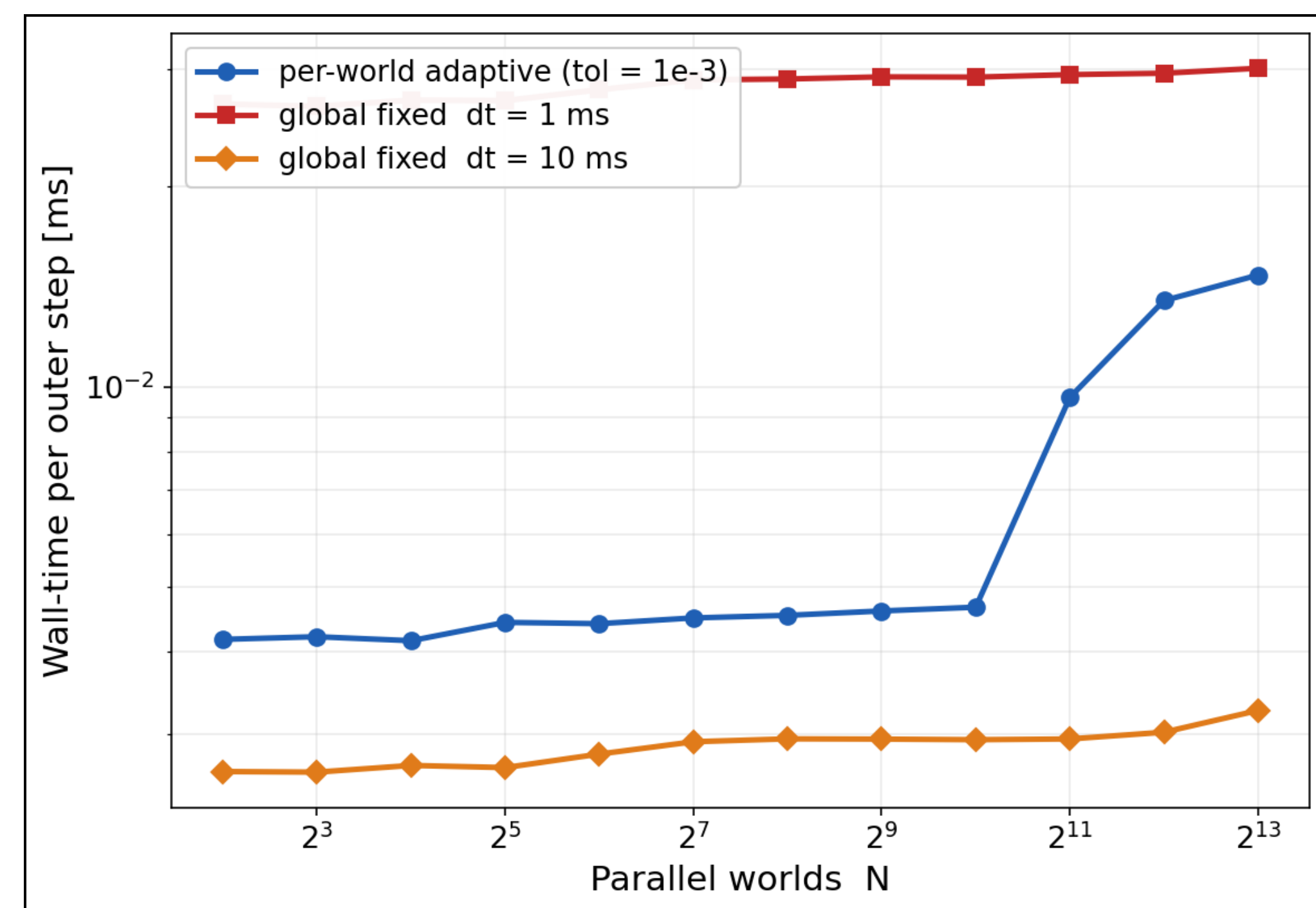


Figure 1: **Wall-time scaling**. Per-world adaptive vs fixed- δt , $N = 4$ to 8192 worlds.

OUR CONTRIBUTION

We bring convex error-controlled integration¹ **from the CPU to massively parallel GPU hardware**. Each of N worlds advances its **own inner δt** between fixed outer control boundaries, with step acceptance, error estimation, and step-size update all running on-device. The only host sync is a 4-byte boundary flag per iteration; no per-state PCIe transfers in the physics loop.

¹Convex error-controlled integration (CENIC), *arXiv:2511.08771*.

PER-WORLD ADAPTIVE TIME STEPPING

The system evolves continuously according to

$$x(t_{n+1}) = x(t_n) + \int_{t_n}^{t_{n+1}} f(\tau, x(\tau)) d\tau,$$

which the integrator approximates over $\delta t = t_{n+1} - t_n$. Each of the N parallel worlds carries its own state $x^{(i)}$ and its own inner step $\delta t^{(i)}$, advancing independently between fixed outer control boundaries.

STEP DOUBLING

Let $\Phi_{\delta t}(x_n)$ denote one MuJoCo Warp convex step. Compare a full step against two successive half-steps:

$$\hat{x}_{n+1} = \Phi_{\delta t}(x_n), \quad (1)$$

$$x_{n+1} = \Phi_{\delta t/2}(\Phi_{\delta t/2}(x_n)). \quad (2)$$

The step-doubling difference yields a second-order error estimate

$$e_{n+1} = \|S(q_{n+1} - \hat{q}_{n+1})\|_{\infty} \approx \bar{c} \delta t^p, \quad p = 2, \quad (3)$$

a weighted, position-only inf-norm per world with $S = \text{diag}(M)^{-1/2}$.

STEP-SIZE SELECTION

Accept if $e_{n+1} \leq \epsilon_{\text{acc}}$ (or the controller grows δt); otherwise reject and retry. Update the step from the asymptotic error model:

$$\delta t_{\text{new}} = k_{\text{safe}} \delta t \left(\frac{\epsilon_{\text{acc}}}{e_{n+1}} \right)^{1/p}, \quad \delta t = \min(\delta t_{\text{new}}, k_{\text{grow}} \delta t, t_{\text{outer}} - t_n).$$

Constants: $k_{\text{safe}} = 0.9$, $k_{\text{grow}} = 5.0$. The boundary clamp prevents fast worlds from overshooting, while slow worlds catch up. A symmetric deadband $[0.9, 1.2] \delta t$ and shrink cap $k_{\text{shrink}} = 0.1$ suppress step-size thrash.

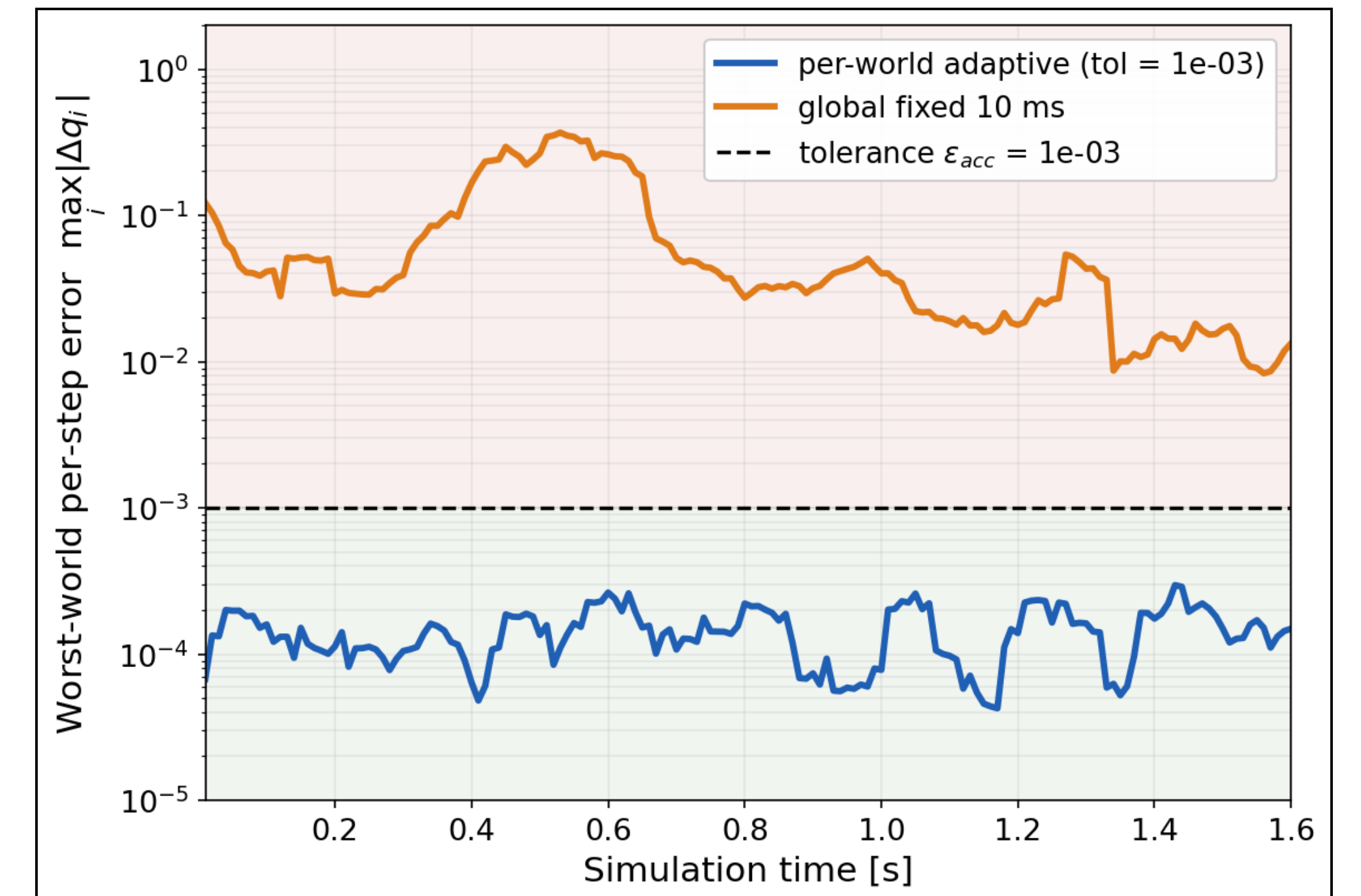


Figure 4: **Work-precision**. Per-world δt beats global worst-case δt across all tolerances.

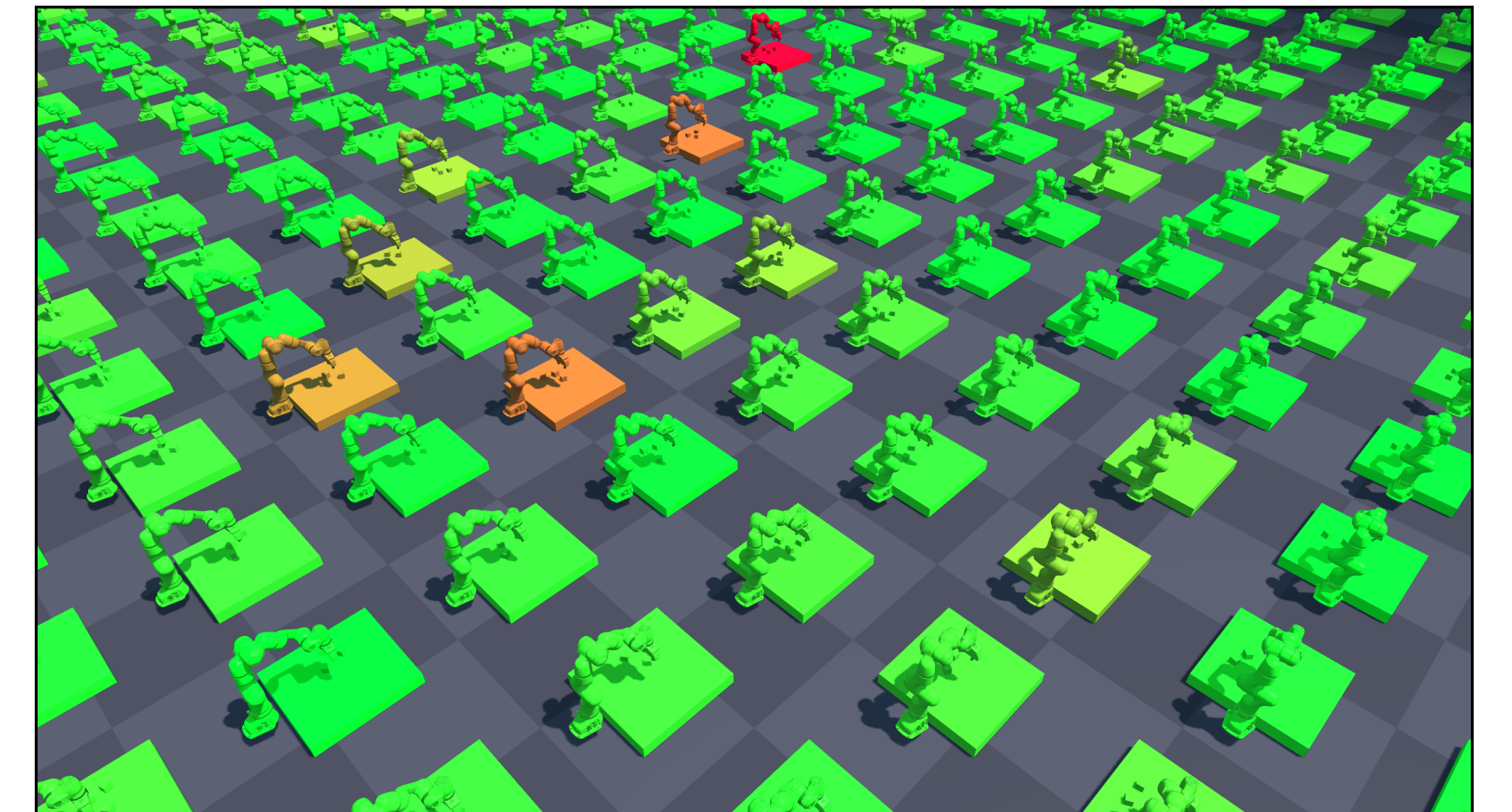


Figure 5: **1024 worlds in parallel**. Franka arms stepped simultaneously on one GPU, each tinted by its per-world adaptive δt (green = large step / free motion, red = small step / stiff contact). Worlds diverge and each finds its own δt .

KEY RESULTS

- Integration error held inside user-specified tolerance
- Contact artifacts eliminated at comparable wall-time budgets

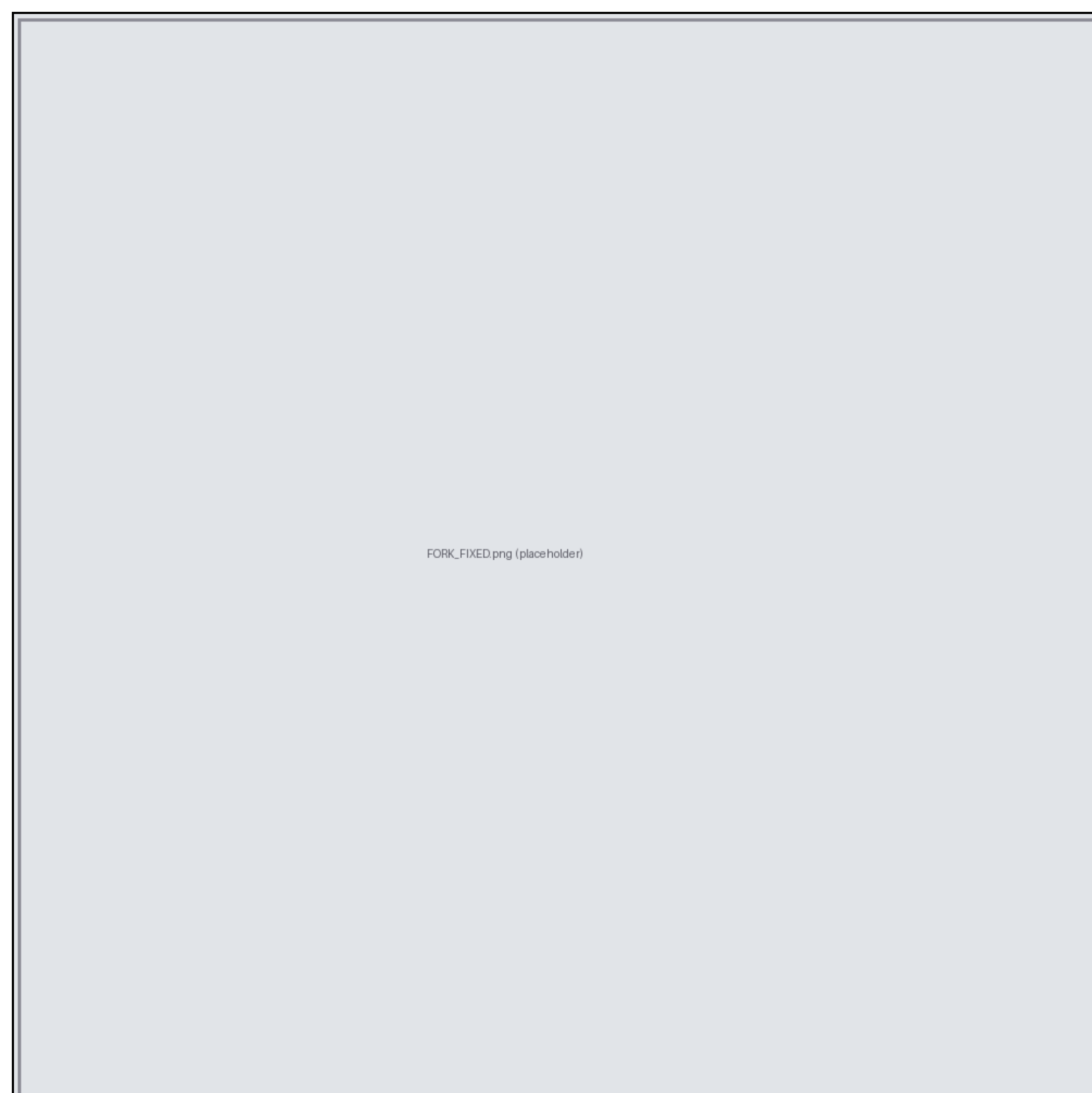


Figure 2: **Fixed step**, $\delta t = 10$ ms. Fork penetrates rack.

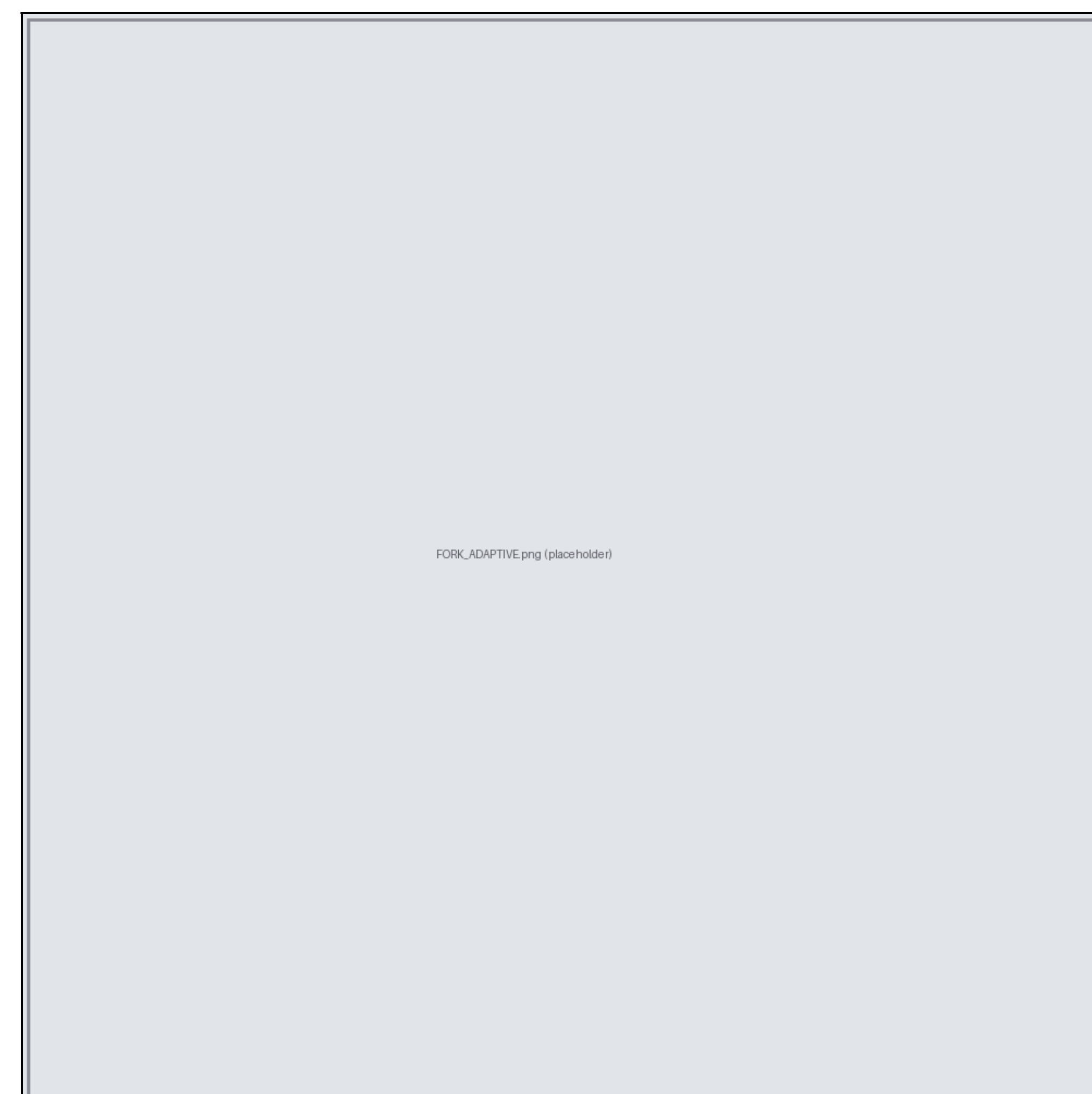


Figure 3: **Adaptive step**, $\epsilon_{\text{acc}} = 10^{-4}$. Fork rests on rack.